Table of contents

An example notebook 1
Going further 5

An example notebook

If you want to see an example of documented code, have a look at a notebook that I wrote for the analysis we do in this tutorial. You can think of it as a cleaned-up version of the `notes.md` notebook that you kept during the workshop.

The link above shows you:

- How you can use GitHub to make your code available to others
- How you can organize a project folder, with a `README.md` file that explains what is in the folder and where to start
- What a code notebook can look like. The notebook is a Quarto file, which you find here. To make it easier to read for collaborators, I also turned it into an HTML report.

In the example, the notebook has its own folder, `notebook/`, next to the `scripts/` folder. In the tutorial, you kept your `notes.md` directly in `data_analysis/`. Both work. A separate folder keeps a bigger project tidy, but all commands still run from the project folder itself.

The example uses the same data as this tutorial, so the code might only make sense after you have finished the tutorial. Feel free to come back to this page after you have written some code yourself.

This is just one example to get you started. Your own notebook might look different, depending on your needs.

Below, you find some hints on what to put into the different sections of the notebook.

YAML headers

```yaml
---
title: "My report  for analysis X (project title)"

author: Nina Dombrowski
date: 12/18/2023

format:
  html:
    embed-resources: true

toc: true
toc-depth: 2

execute:
  eval: false

engine: knitr
---
```

The text between the `---` is what is called a YAML header an
completely optional but useful if you document code in Rma
or Quarto.

It can be used to specify:
- The characteristics of your document, i.e. title, authors, dat
creation
- Arguments that control how you render your document, fo
example in this example to HTML
- Other parameters for your report. For example, I do not wa
code cells to executed (eval: false)

The **YAML header** is the part between the two `---` lines at the very top of the file. It contains settings for the whole document, such as the title, the author and the output format. In my example, `eval: false` tells Quarto to only show the code, not to run it.

Settings (change these before you run the workflow)

```bash
# Path to working directory
wdir="$HOME/personal/data_analysis"

# Custom variables
download_link="https://raw.githubusercontent.com/ndombrowski/cli_workshop/main/data/seq_project.tar.gz"
```

In one of the first sections of my notebooks, I collect everything that someone who wants to use my workflow has to change. I write the code below this section in a way that it does not need to be changed. This way, another person can run my code as it is, and only needs to edit this first section. Typical things to add here are:

- Where the analysis starts: the location of the working directory or project folder
- Custom paths, for example to databases that need to be downloaded from the web
- Custom variables. In the example above, I store the location of my project folder in a variable called `wdir`. I use `$HOME` in it instead of typing out my home directory, so the path works for anyone with the same folder structure on Crunchomics. I also store the link to the data in a variable called `download_link`, and use this variable further down to download the data. If the link to the data ever changes, another person only needs to change it in this one place, and the code below stays the same. When you

write code, try to think ahead: keep the number of places that need to be changed as small as possible.

I tend NOT to write down how I log in to the HPC with `ssh` and copy data with `scp`. That way, my user name and login information stay private.

Looking at the data (sanity checks)

```
# We work with 8 files from 4 samples
wc -l fastq_files.txt
wc -l samples.txt

# We have as many R1 as we have R2 files
ls data/seq_project/*/*R1* | wc -l
ls data/seq_project/*/*R2* | wc -l

# The sequence header looks as expected
zcat data/seq_project/*/*gz | head

# R1 and R2 reads have the same number of reads
mkdir -p results/counts

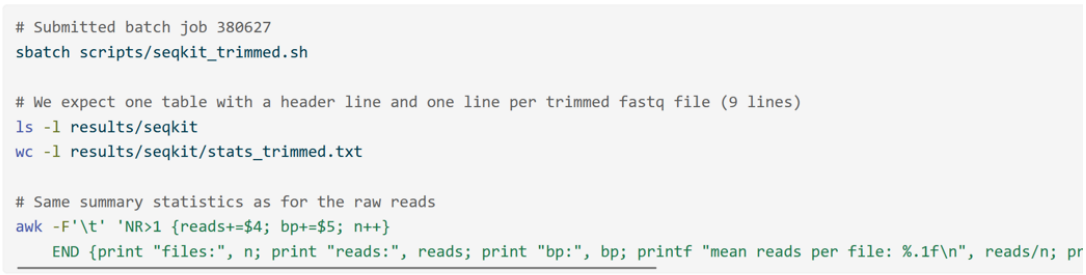
for fastq_file in $(cat fastq_files.txt); do
    echo "$fastq_file: $(( $(zcat data/seq_project/*/$fastq_file | wc -l) /4 ))"
done > results/counts/read_counts.txt
```

When I start working with new data, I add as many sanity checks as possible, to make sure that my data looks good. That way, I notice problems early, before they cause trouble further down the line. At the same time, I get to know my data better: how many samples and how much data I am working with.

I also add sanity checks whenever I change my data. For example, when I merge several files into one large file, I might count the lines of the individual files and of the combined file, to make sure that I used my wildcards correctly.

Remember: the computer is only as smart as the person using it, and it runs your commands blindly. So the computer can do unexpected things, and you need to check for that.

Using Markdown to make notes



```
# Submitted batch job 380627
sbatch scripts/seqkit_trimmed.sh

# We expect one table with a header line and one line per trimmed fastq file (9 lines)
ls -l results/seqkit
wc -l results/seqkit/stats_trimmed.txt

# Same summary statistics as for the raw reads
awk -F'\t' 'NR>1 {reads+=$4; bp+=$5; n++}
    END {print "files:", n; print "reads:", reads; print "bp:", bp; printf "mean reads per file: %.1f\n", reads/n; pr
```

Comments on the seqkit output post trimming:

- After cleaning we work with
  - 214,906 out of 217,486 reads (98.8% maintained)
  - 53,826,288 out of 54,493,987 bp (98.8% maintained)
- Information from the fastp html:
  - Little adapter sequence signal (< 0.05%)
  - Read quality drops at the beginning and end of the read –> reconsider re-running fastp and use `--cut_front`, `--cut_front_window_size 4` (default setting) and `--cut_front_mean_quality 20` (default setting). Note that `--cut_front` is disabled by default

In the example above, I use Markdown not only to document my code, but also to add notes wherever I need them. Here, I wrote down what the results of seqkit stats and the fastp HTML reports actually told me.

This kind of documentation can be useful for:

- Justifying decisions further down the line. In this example, I might decide on how to clean the sequencing data. For example, when I looked at the FastP HTML report I noticed a dip in quality at the end of the sequence reads. Based on that I normally would make a decision to adjust re-run FastP and enable trimming of the reads.
- Future you. If you read the report a month, or a year, later you have the key information in your report and don't have to open any files or tables that are located elsewhere

Documenting external scripts

```
mkdir -p results/seqkit

# Submitted batch job 380625
sbatch scripts/seqkit.sh

# We expect one table with a header line and one line per fastq file (9 lines)
ls -l results/seqkit
wc -l results/seqkit/stats_raw.txt
cat results/seqkit/stats_raw.txt
```

This part makes more sense after you have worked through the HPC part of the tutorial. Here, my notebook only says that I submitted a script to the HPC with `sbatch`. It does not say how I ran seqkit: that code is "hidden" inside the `seqkit.sh` script. So someone who reads only the notebook does not see the code right away. You can deal with this in two ways:

1. Instead of the `sbatch` command, write the actual command that ran on the compute node into your notebook.
2. Keep the scripts next to your notebook, and when you publish your code with a paper, publish the whole `scripts` folder too, for example on GitHub or Zenodo. This is why you copied the job scripts to your own computer at the end of day 2.

I prefer option 2, because I like to record the code in my notebooks exactly as I ran it. But you can do it differently, as long as all the code you ran is written down and available to others once you publish your data.

Going further

From `notes.md` to a Quarto document

Your `notes.md` and my example notebook are written in the same way: Markdown text with code blocks. The difference is that the example notebook is a **Quarto** document (it ends in `.qmd`). Quarto turns such a file into a web page (HTML), a PDF or a Word file, with a table of contents and colored code. That makes it easier to share your notebook with others, for example with your supervisor or together with a paper.

If you want to try this in RStudio (recent versions of RStudio come with Quarto):

1. Go to **File**, then **New File**, then **Quarto Document**
2. Copy the content of your `notes.md` into the new file, below the YAML header at the top (see "YAML headers" above)
3. Save the file and click **Render**

Code blocks that you wrote as ```` ```bash ````, like in your notes, are only shown on the page, not run. The Quarto tutorial for RStudio explains the next steps.

Further reading

- A Guide to Reproducible Code in Ecology and Evolution from the British Ecological Society explains how to organize project folders and how to write clear code that others can rerun. The examples are mostly in R, but the advice applies to bash and other languages as well.
- Markdown Basics on the Quarto website lists everything else you can do with Markdown, for example lists, links, bold text, images and tables.